

ANNDL EXAM CHEAT SHEET

Politecnico di Milano · Boracchi & Matteucci · Skim before & during prep

1 · PARAMETER CALCULATION (Q10 — 2.5 pts)

The Formulas

CONV2D params =
 $(k \times k \times C_{in} + 1) \times C_{out}$

DENSE/LINEAR params =
 $(in_features + 1) \times out_features$

BATCHNORM params = $4 \times C$
(2C trainable: γ, β) + (2C non-train: mean, var)

0 params: ReLU, Tanh, Sigmoid, MaxPool, AvgPool, Dropout, Upsample, Flatten, Softmax, GAP, Add, Concatenate

Output Spatial Size

$out = \text{floor}((in + 2p - k)/s) + 1$

k	pad	stride	H,W effect
3	1	1	SAME
5	2	1	SAME
1	0	1	SAME
3	0	1	-2
3	1	2	HALVE

MaxPool 2×2 → HALVE H,W · **Upsample ×2** → DOUBLE H,W (C unchanged for both)

#1 TRAP: `torch.cat([a,b], dim=1)` ADDS channels (U-Net). Next conv C_{in} = sum! · `torch.add` / `a+b` keeps channels same (ResNet). Always read **forward()**, mark every cat/add.

LeNet-style Steps (verified example, input 3×256×256, U-Net w/ skips)

Layer	Calc	Out (C,H,W)	Params
Conv2d(3,32,k3,p1)	$(3 \cdot 3 \cdot 3 + 1) \cdot 32$	32,256,256	896
MaxPool2	—	32,128,128	0
Conv2d(32,64,k3,p1)	$(3 \cdot 3 \cdot 32 + 1) \cdot 64$	64,128,128	18,496
MaxPool2	—	64,64,64	0
Conv2d(64,128,k3,p1)	$(3 \cdot 3 \cdot 64 + 1) \cdot 128$	128,64,64	73,856
Upsample×2	—	128,128,128	0
cat w/ 64ch skip	$128 + 64 = 192$	192,128,128	—
Conv2d(192,64,k3,p1)	$(3 \cdot 3 \cdot 192 + 1) \cdot 64$	64,128,128	110,656
Upsample×2	—	64,256,256	0
cat w/ 32ch skip	$64 + 32 = 96$	96,256,256	—
Conv2d(96,32,k3,p1)	$(3 \cdot 3 \cdot 96 + 1) \cdot 32$	32,256,256	27,680
Conv2d(32,16,k1)	$(1 \cdot 1 \cdot 32 + 1) \cdot 16$	16,256,256	528
Conv2d(16,8,k1)	$(1 \cdot 1 \cdot 16 + 1) \cdot 8$	8,256,256	136
Conv2d(8,3,k1)	$(1 \cdot 1 \cdot 8 + 1) \cdot 3$	3,256,256	27
TOTAL			232,275

2 · RECEPTIVE FIELD (work BACKWARDS)

Start RF=1 at target. Go backward:

- Conv $k \times k \rightarrow \text{RF} + (k-1)$
- Pool $s \times s \rightarrow \text{RF} \times s$
- ReLU/BN/Dropout/Linear $\rightarrow$ no change

Example: Conv5→MP2→Conv3→MP2→Conv1→BN

start=1 Conv1: $1+0 = 1$ MP2: $1 \times 2 = 2$ Conv3: $2+2 = 4$
MP2: $4 \times 2 = 8$ Conv5: $8+4 = 12 \rightarrow 12 \times 12$

Shortcuts: two 3×3 = one 5×5 · three 3×3 = one 7×7 · each
MaxPool DOUBLES RF · 1×1 conv = no change

3 · LSTM (Q5 — 3.5 pts) · "Long Short-Term Memory"

#	Equation	One-sentence meaning
1	$i_t = \sigma(\dots)$	Input gate: how much NEW info to let into cell state
2	$C_t = \tanh(\dots)$	Candidate: new candidate values that could be added to memory
3	$f_t = \sigma(\dots)$	Forget gate: how much OLD cell state to keep (1) or forget (0)
4	$C_t = f_t \cdot C_{(t-1)} + i_t \cdot C_t$	Cell update: new memory = kept old + filtered new candidate
5	$o_t = \sigma(\dots)$	Output gate: what part of cell state to expose
6	$h_t = o_t \cdot \tanh(C_t)$	Hidden state: output = squashed cell state filtered by output gate

CEC (Constant Error Carousel): cell state path is ~linear → gradient ≈ 1 → prevents vanishing gradient. **NOT permutation invariant** (unless sorted first). Bidirectional = look-ahead (spellcheck, sentiment).

4 · SUPERVISED vs UNSUPERVISED

Network/Task	Type	Why
Autoencoder	UNSUP	reconstructs its own input, no labels
GAN	UNSUP	generates from noise, no labels
Word2Vec / embeddings	UNSUP	self-supervised from text context
Anomaly detection	UNSUP	usually trained on normal data only
PCA / clustering	UNSUP	no target labels
Classification CNN	SUP	needs class labels
Regression net	SUP	needs target values
YOLO / R-CNN / detection	SUP	needs bounding boxes + labels
U-Net / segmentation	SUP	needs pixel-level masks
Mask R-CNN	SUP	needs masks + boxes + labels

5 · GENERALIZATION vs TRAINING-PERFORMANCE

(aka "Overfitting" vs "Backpropagation" — SAME answers, different names)

GENERALIZATION (fights overfitting)	PERFORMANCE (helps backprop/convergence)
Early stopping	ReLU / Leaky ReLU
Dropout	Skip / shortcut connections
Weight decay (L2)	Batch normalization
Data augmentation	Xavier / Glorot initialization
	Learning rate / momentum

Trick: "limits/regularizes the model" → Generalization. "helps gradients flow / converge" → Performance.

6 · LOSS FUNCTIONS PER TASK

Task	Loss	Output activation
Multi-class classification	Categorical Cross-Entropy	Softmax
Binary / multi-label classification	Binary Cross-Entropy	Sigmoid
Regression	MSE (or MAE)	Linear (none)
Semantic segmentation	Cross-Entropy (per pixel)	Softmax per pixel
Autoencoder / denoising	MSE $\ x - g(x)\ ^2$	Linear/Sigmoid
Object detection (loc+class)	Multi-task: MSE + Cross-Entropy	mixed
Word2Vec	Cross-Entropy	Softmax

7 · "WHAT TASK CAN THIS NET DO?" (Q11)

Step 1: output shape · Step 2: loss · Step 3: match

Output + Loss	Task	Example
(C,H,W) same as input + MSE	Per-pixel regression / image-to-image	rainfall/snow/UV per pixel; denoising; colorization
(C,H,W) per-pixel + Cross-Entropy	Semantic segmentation	label each pixel road/car/sky
(num_classes,) + softmax + CE	Image classification	Horse/Shark/Bird (ONE label)
(1,) + sigmoid + BCE	Binary classification	real vs fake
(N,) + linear + MSE	Regression (whole image)	building height/width/depth
2 heads: class + 4 box coords	Localization	1 object + box

Traps: per-pixel net CANNOT do image-level classification/counting. MSE loss ≠ segmentation (needs CE). Always check output shape AND loss together.

8 · KEY CONCEPTS (open-question ammo)

IR (Information Retrieval): find DB items most similar to a query. CNN: remove classifier → keep Feature Extraction Net (FEN) → compute latent vector per image → retrieve K-nearest neighbors in latent space (L1/L2/cosine). Latent distances are semantically meaningful, unlike pixels. (Like Google Image Search.)

Autoencoder: UNSUP net. Encoder → bottleneck → Decoder reconstructs input. Loss=MSE. Bottleneck forces learning structure. Uses: compression, denoising, anomaly detection, feature extraction, generation (VAE).

GAN = Generative Adversarial Network. Generator (noise → fake) vs Discriminator (real/fake). Adversarial training, UNSUP, unstable (dropout helps). After training, discard Discriminator. Generates high-res faces.

CAM (Class Activation Map): heatmap of where the net looks. Needs GAP + single dense layer. $M_c(x,y) = \sum_k w_k^c \cdot f_k(x,y)$. Different weights per class → class-specific. **Grad-CAM:** uses gradients instead → works on ANY CNN, no architecture change.

R-CNN family: R-CNN (Selective Search, CNN per region, 47s) → Fast R-CNN (CNN once + ROI pooling, joint loss) → Faster R-CNN (learned RPN with anchors+objectness, 0.2s). YOLO=single-shot (grid, no proposal stage). Mask R-CNN=Faster R-CNN + small mask branch (NOT U-Net per box).

Convolution facts: linear classifier = correlation vs templates. Conv = linear ONLY (ReLU separate). Bias = 1 per filter. Stacked linear convs = 1 conv. Conv = sparse matrix, shared weights. Translation invariant.

9 · EXAM-DAY: negative marking strategy

✓ 90%+ sure → **SELECT**. ✗ 50/50 → **LEAVE BLANK** (blank=0, wrong=negative). Absolute words ("only/always/never") usually FALSE. Order of attack: Q10 calc → Q5 LSTM → Q1 table → Q11 task → then MCQs. Need ~11/21 to pass. Stay calm & methodical.